

HO CHI MINH UNIVERSITY OF TECHNOLOGY

ELECTRICAL ELECTRONIC ENGINEERING

Electronic Department

-----o0o-----



LOGIC DESIGN / LOGIC SYNTHESIS

Reverse Polish Notation Calculator

INSTRUCTORS: Tran Hoang Linh

Nguyen Tuan Hung

STUDENT : Bui Nguyen Thanh

Long _ 2051140

Pham Gia Khanh _ 2051132

HO CHI MINH CITY, APRIL, 2025

Table of Contents

Contents

I. OVERVIEW	3
II. THEORY.....	4
III. DESIGN STRATEGY	6
V. IMPLEMENTING ON HARDWARE	11
VI. CONCLUSION.....	15
VII. REFERENCES	15

I. OVERVIEW

The purpose of this project is to use what we have learn in previous lab to make something fun. We decide most of the specification in this project. We are going to make a Reverse Polish notation calculator. Every calculators that has a C(clear) button, input as pressing a number then press "+ -" then press another number then press "=" is this RPN Calculator.

But the original calculator has a translator, because RPN works like this. "3 4 5 + - =" is "3 + 4 - 5 =". So we need to make that translator too. The real calculators has buttons, DE10 has buttons but much less. So let's improvise an interface.

- KEY0 is Reset/Clear
- KEY1 is Input
- The LEDR0 will light when it's showing the result.
- The LEDR1 will light when you input something illegal like "999" or a "NULL" command.
- SW0 to SW3 is recognized as follows when you press the input button

Binary	Value	Binary	Value	Binary	Value	Binary	Value
0000	0	0100	4	1000	8	1100	NULL
0001	1	0101	5	1001	9	1101	NULL
0010	2	0110	6	1010	“+”	1110	“=”
0011	3	0111	7	1011	“-”	1111	NULL

The 7segments LED HEX0 to HEX5 will display the number we JUST input (after we press the input). It should not display "+,-" or "=". After we input "=" it will display the result. For example, we want to calculate $99 + 102 = ?$ We will press the following buttons:

- SW[3:0] = 1001 (9) and press KEY1 (Display: 9)
- SW[3:0] = 1001 (9) and press KEY1 (Display: 99)
- SW[3:0] = 1011 (-) and press KEY1 (Display: 99)
- SW[3:0] = 0001 (1) and press KEY1 (Display: 1)
- SW[3:0] = 0000 (0) and press KEY1 (Display: 10)
- SW[3:0] = 0010 (2) and press KEY1 (Display: 102)
- SW[3:0] = 1110 (=) and press KEY1 (Display: 201)

II. THEORY

2.1. RPN Converter (Translator)

In reverse Polish notation, the operators follow their operands. For example, to add 3 and 4 together, the expression is 3 4 + rather than 3 + 4. The conventional notation expression 3 - 4 + 5 becomes 3 4 - 5 + in reverse Polish notation. The concept of a stack, a last-in/first-out construct, is integral to the left-to-right evaluation of RPN.

3 + 4

Figure 1. Infix notation

3 4 +

Figure 2. Postfix notation ("Reverse Polish")

2.2. RPN Calculator

Reverse Polish Notation (RPN) is a notation for representing arithmetic formula. The common notations that we are familiar with had the operator placed in between the two operands (called the infix notation), for example, A + B, A * B, A + B + C. The RPN notation (or postfix notation) places the operator at the end of the operands, for example, A B +, A B *, A B + C +. The RPN notation has the advantage that it can express operation precedence without using parentheses. For example,

- Expression in infix notation: (A + B) * (C + D)
- Same expression in postfix notation: A B + C D + *

A stack implementation is typically used to evaluate an expression in RPN. For example, to calculate A B + C D + *. Scanning the RPN expression from left to right. If it is an operand, it is pushed onto the stack. If it is an operator, the top two items on the stack are popped and evaluated with the operator; the result is pushed back onto the stack.

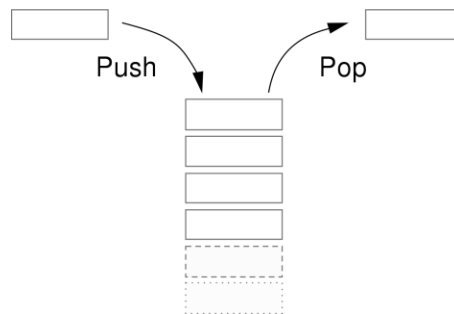
S1 = A + B | S2 = C + D | S3 = S1 * S2

A	B	+	C	D	+	*
A	B	S1	C	D	S2	S3
	A		S1	C	S1	
				S1		

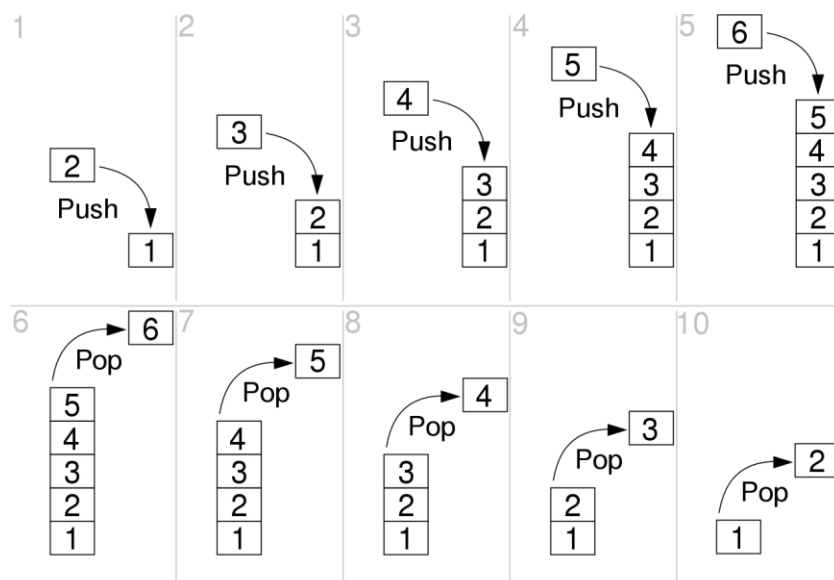
2.3. Stack

A stack is an abstract data type that serves as a collection of elements:

- Push, which adds an element to the collection.
- Pop, which removes the most recently added element.



A peek operation allows for retrieving the value of the last element added to the stack without altering the stack itself. The term "stack" draws an analogy to physical items arranged in a stack, like a stack of plates. The sequence of adding and removing elements from a stack follows the Last In, First Out (LIFO) principle. Similar to a physical stack, this structure facilitates easy removal of the topmost item, while accessing a deeper item may necessitate removing other items first. Defined as a linear data structure, or more abstractly as a sequential collection, a stack has one end designated for push and pop operations - the top of the stack - and a fixed end - the bottom. This structure can be implemented using a singly linked list with a pointer to the top element. A stack can be given a bounded capacity. When the stack reaches full capacity and cannot accommodate additional elements, it enters a state of "stack overflow." A stack is essential for implementing depth-first search algorithms.



III. DESIGN STRATEGY

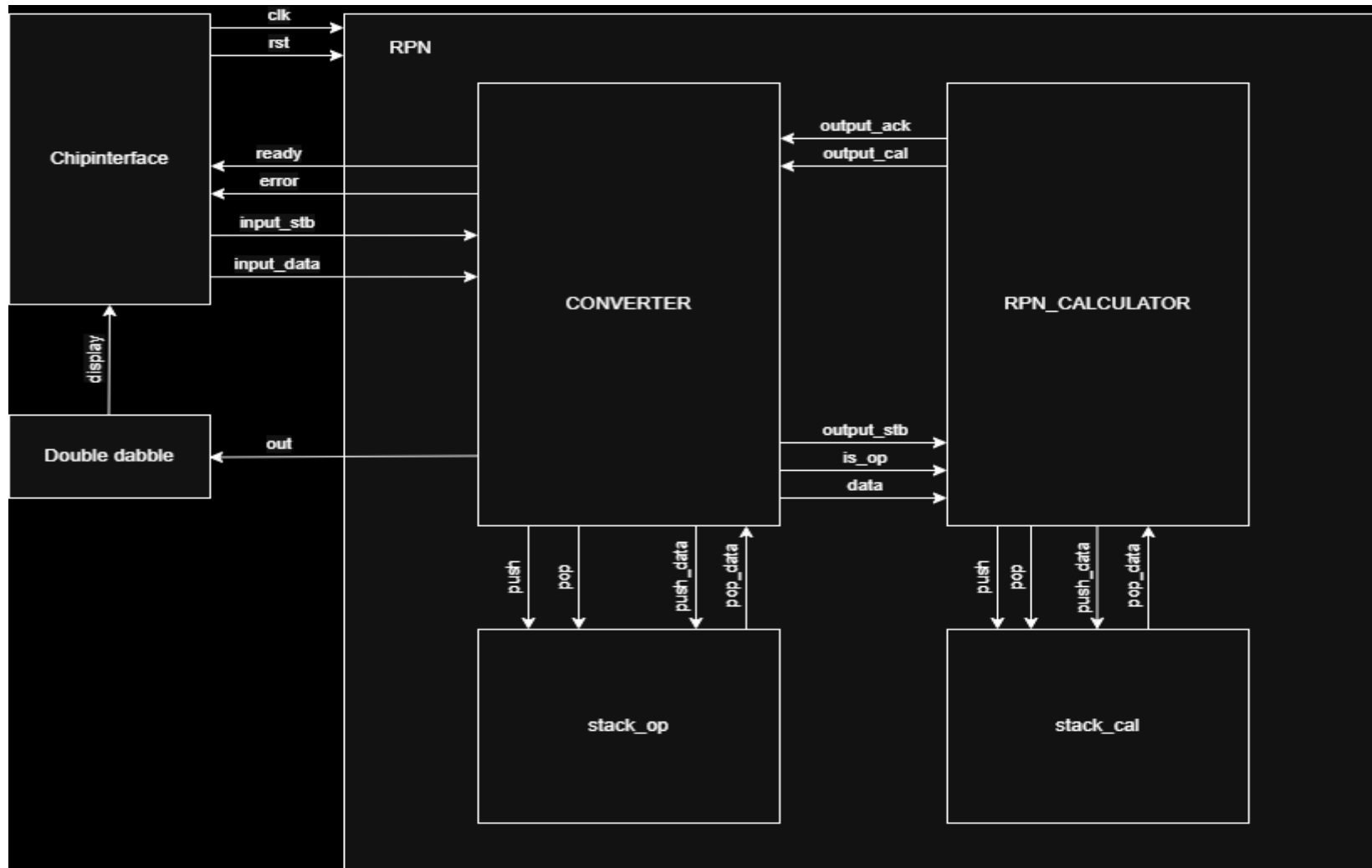


Figure 1: Top module of the design

In this project, we will create 4 components. First, there is a converter to translate the input data from a normal calculator into the format that the rpn_calculator can use. Using the shunting yard algorithm, we will create a stack for the operation input whenever the calculator detects an operation input. It will put the data into the stack, and after that do one of two things: One, it will push the operation into the stack if there hasn't been any operation before it, and the second is to pop out the operation in the stack and wait for that operation to calculate while push the new operation into the stack. The next component is the stack, which will act as a storage for the data and operations to be stored before being used. Furthermore, there is a rpn_calculator that will operate. Every time an operation is pushed into the calculator, it will wait for 2 clock cycles to pop out the data and store it in two different registers, then perform the input operation and push the answer back into the stack. Finally, there is a double dabble module in between the rpn and the chip interface. This module performs some function to turn the output data into a normal decimal code so that we can examine it on the chip more easily.

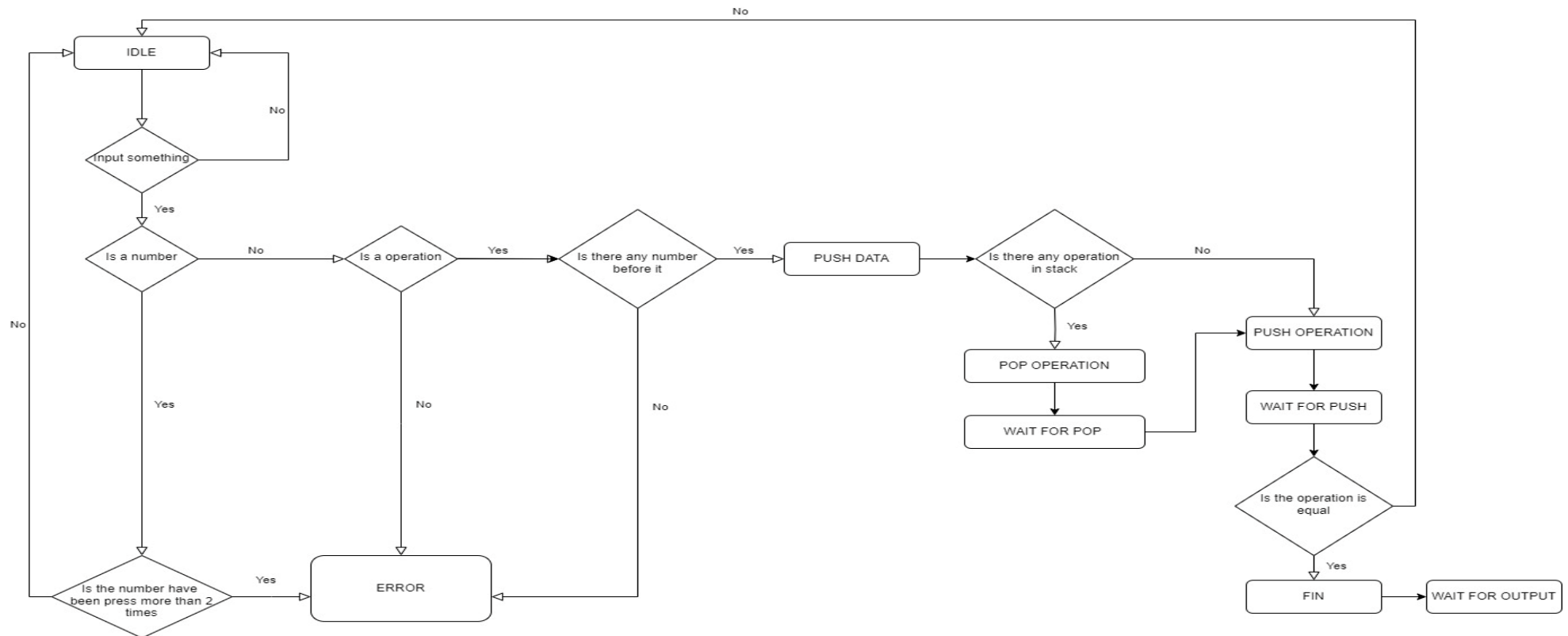


Figure 2: Flowchart of the converter

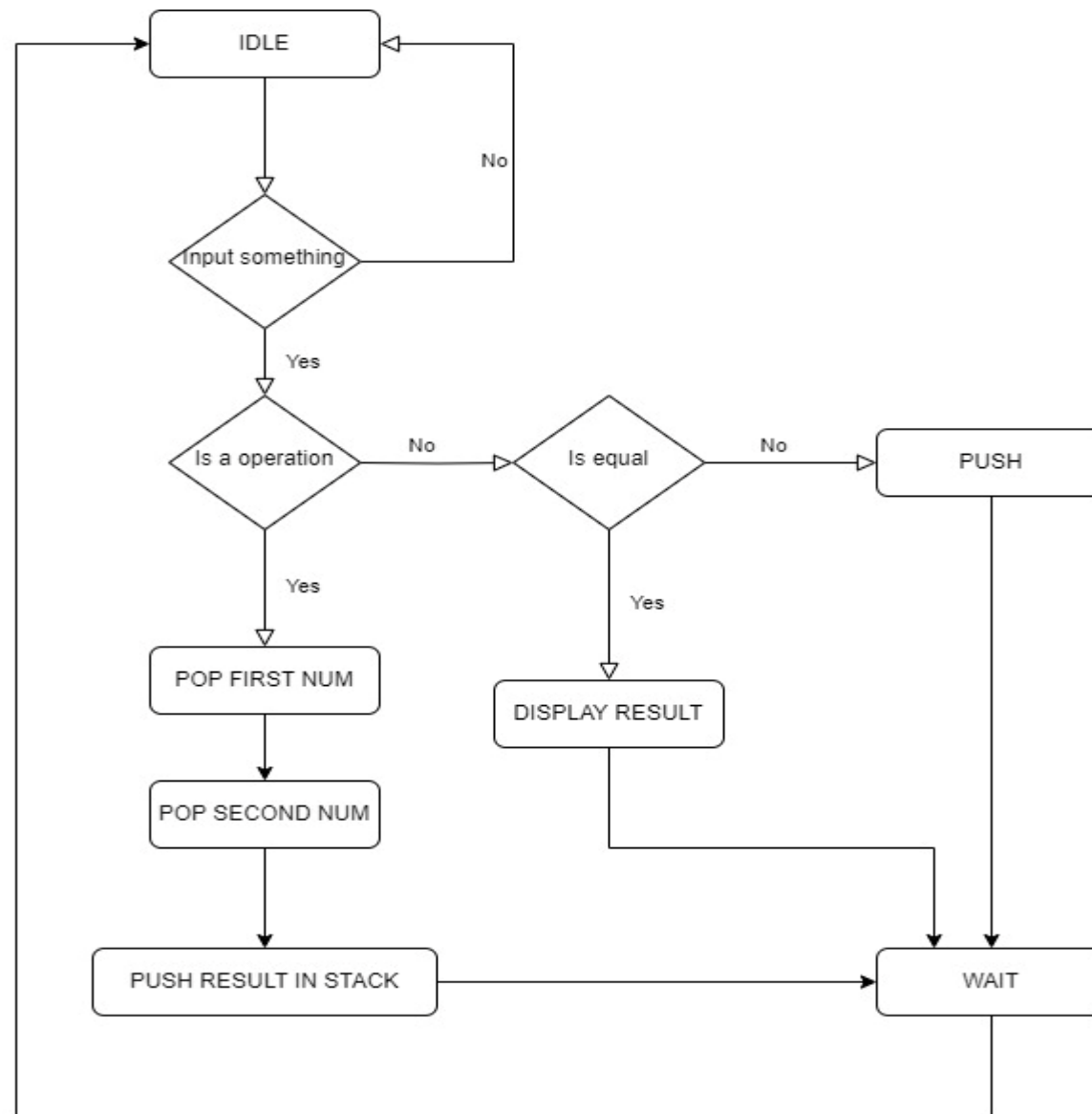
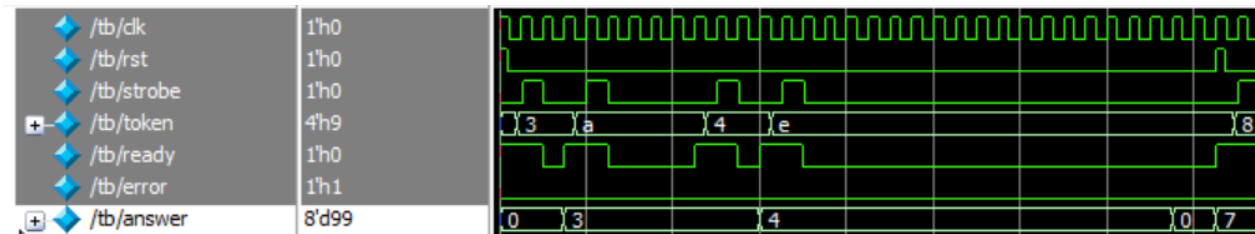


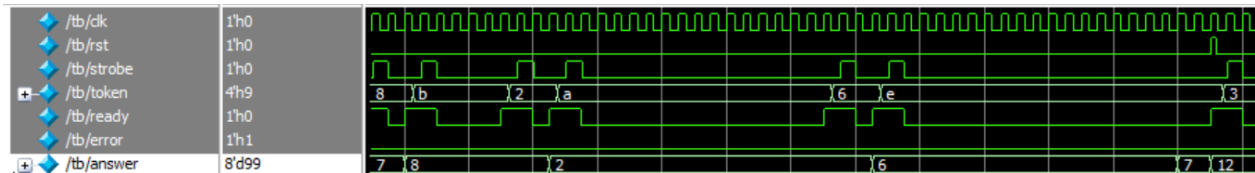
Figure 3: Flow chart of the rpn_calculator

IV. SIMULATING ON QUARTUS

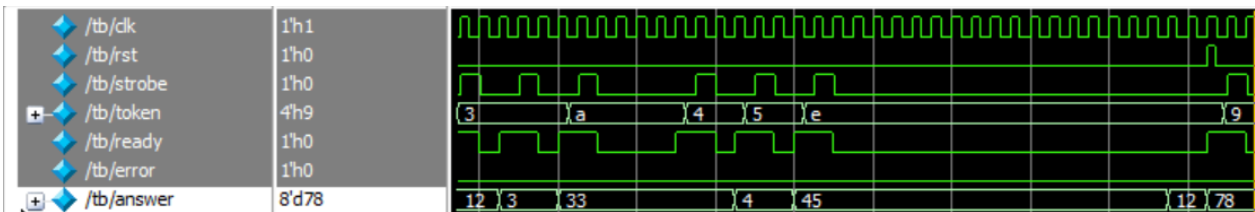
3 + 4 = 7



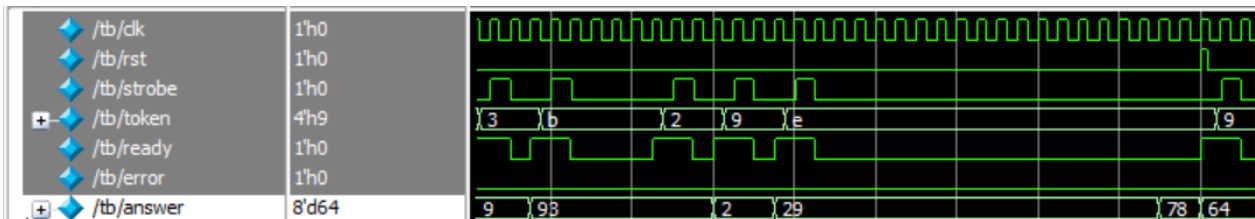
8 - 2 + 6 = 12



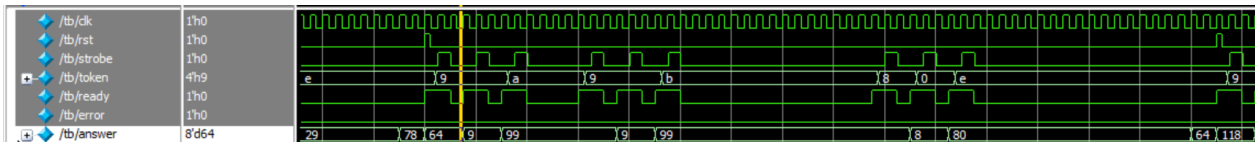
33 + 45 = 78



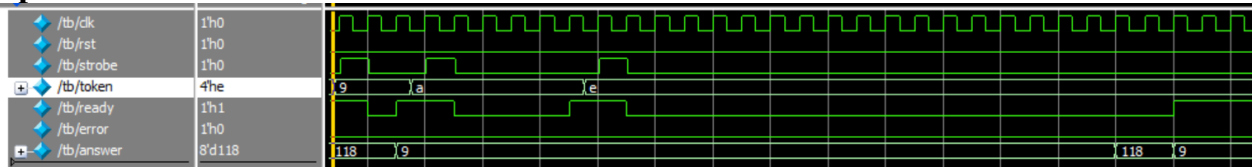
93 - 29 = 64



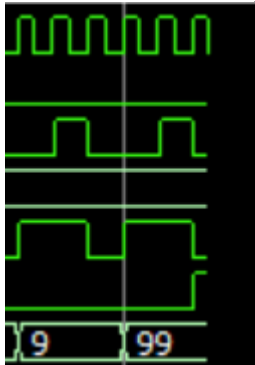
99 + 99 - 80 = 118



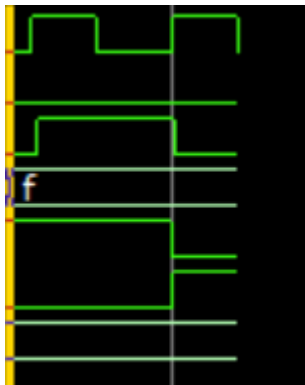
Special case $9 + = 9$



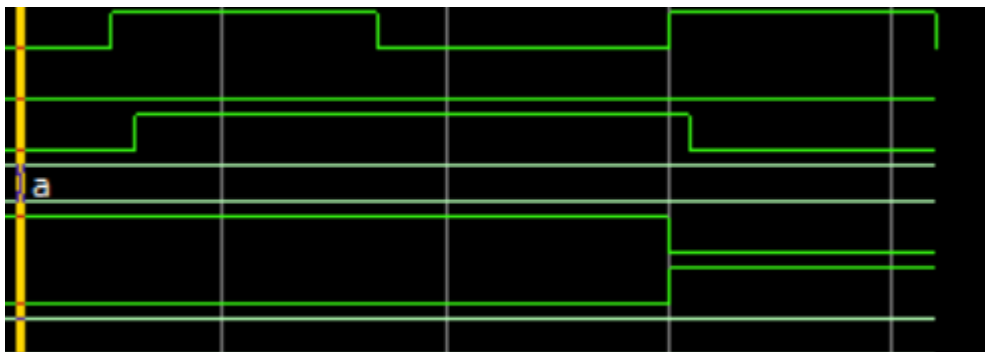
Error case #1: input larger than 99



Error case #2: input is null

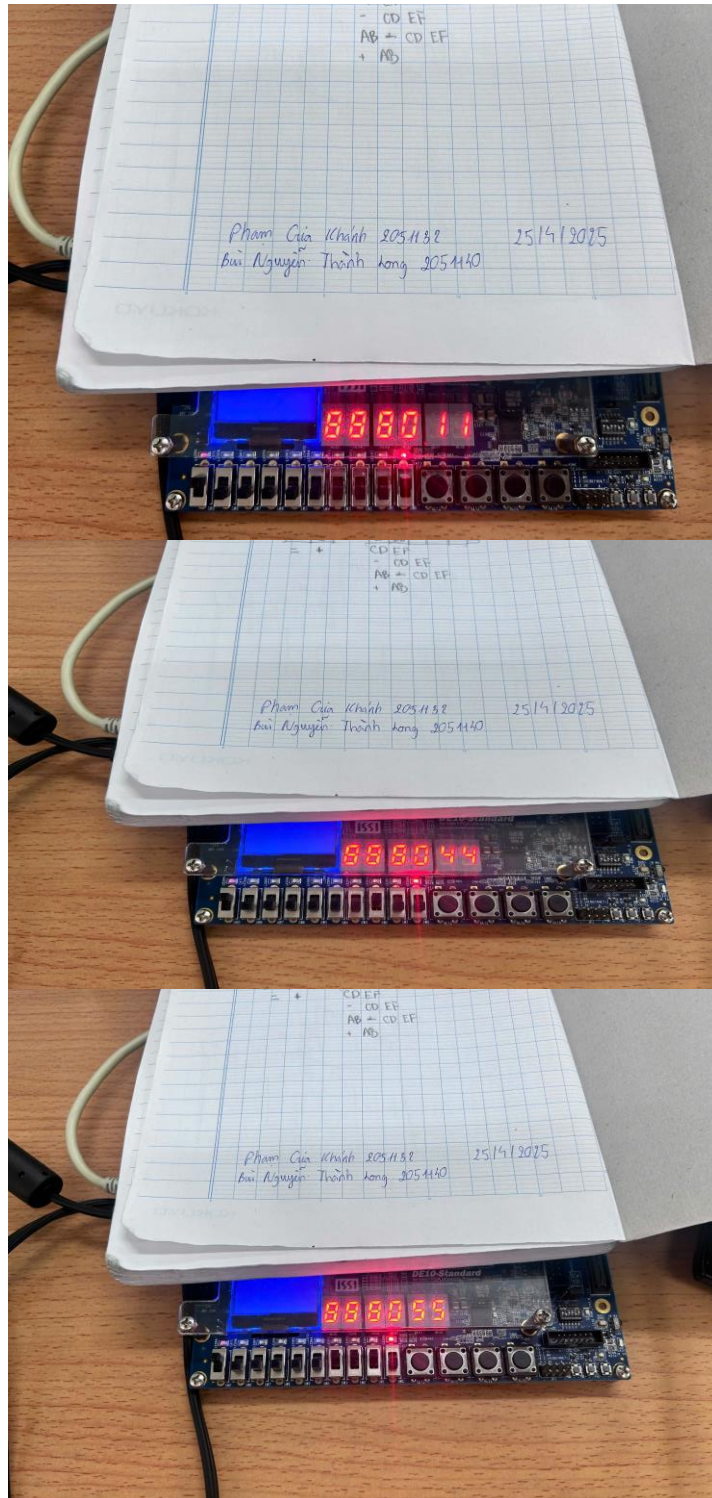


Error case #3: operation before number

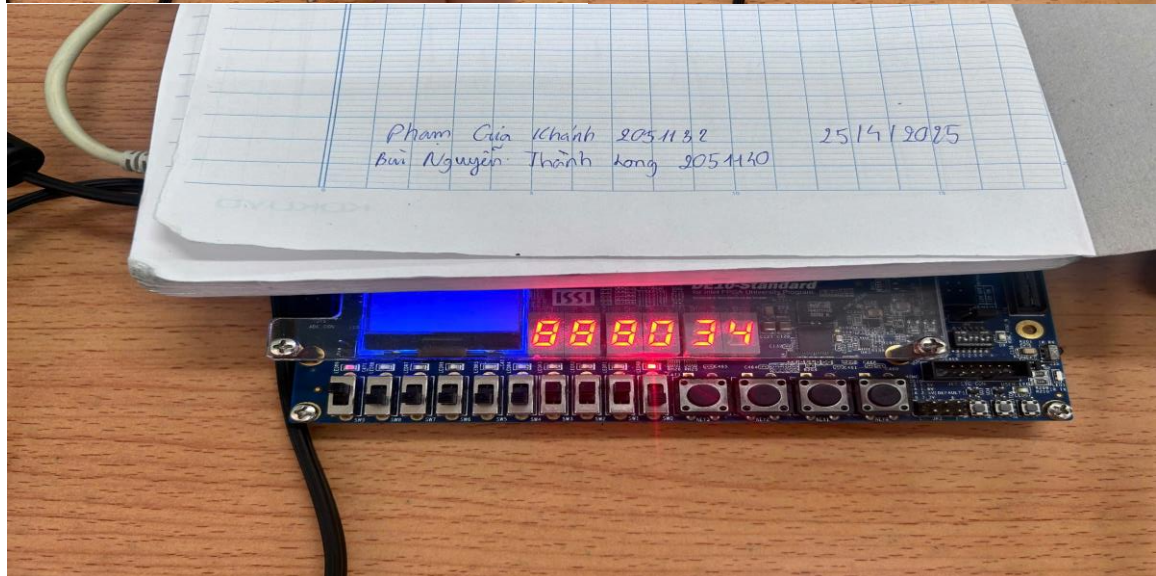
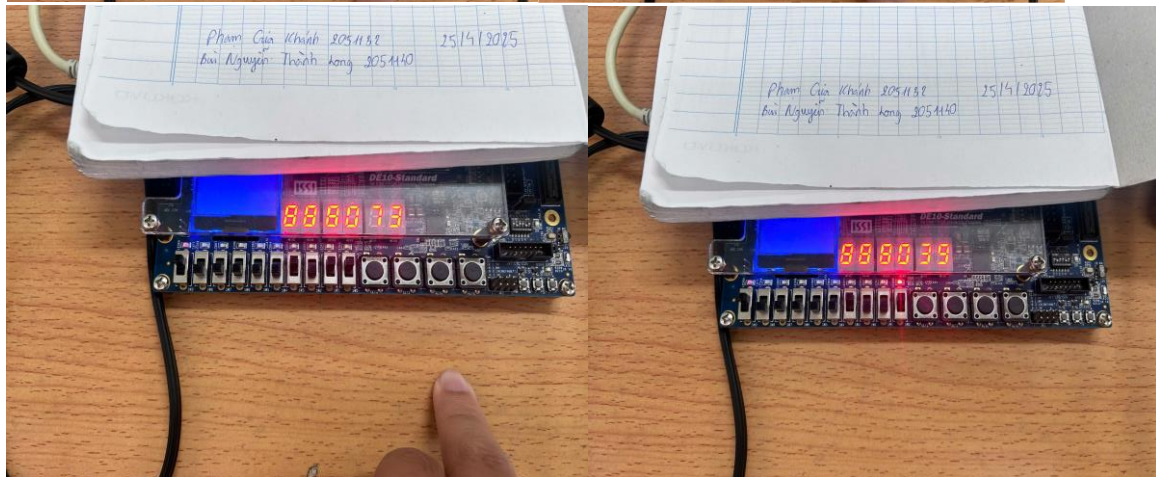
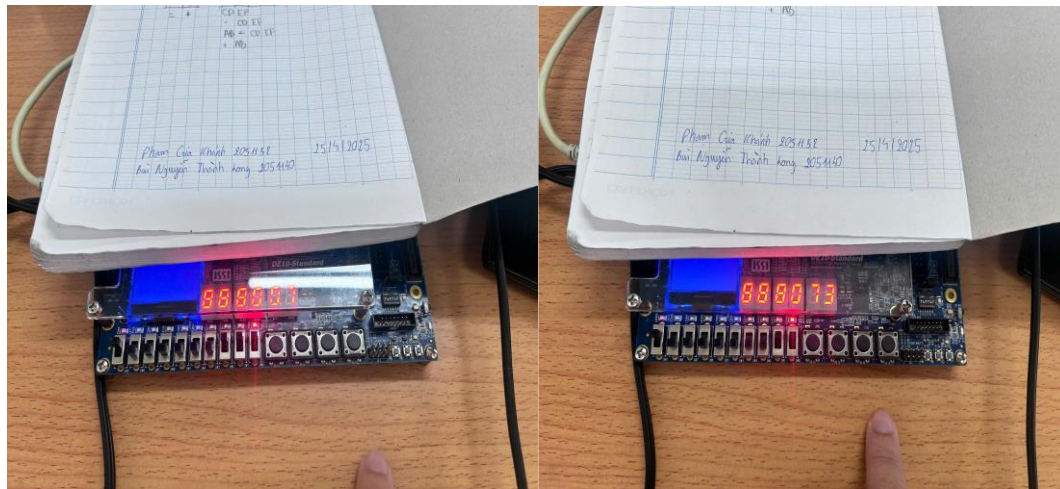


V. IMPLEMENTING ON HARDWARE

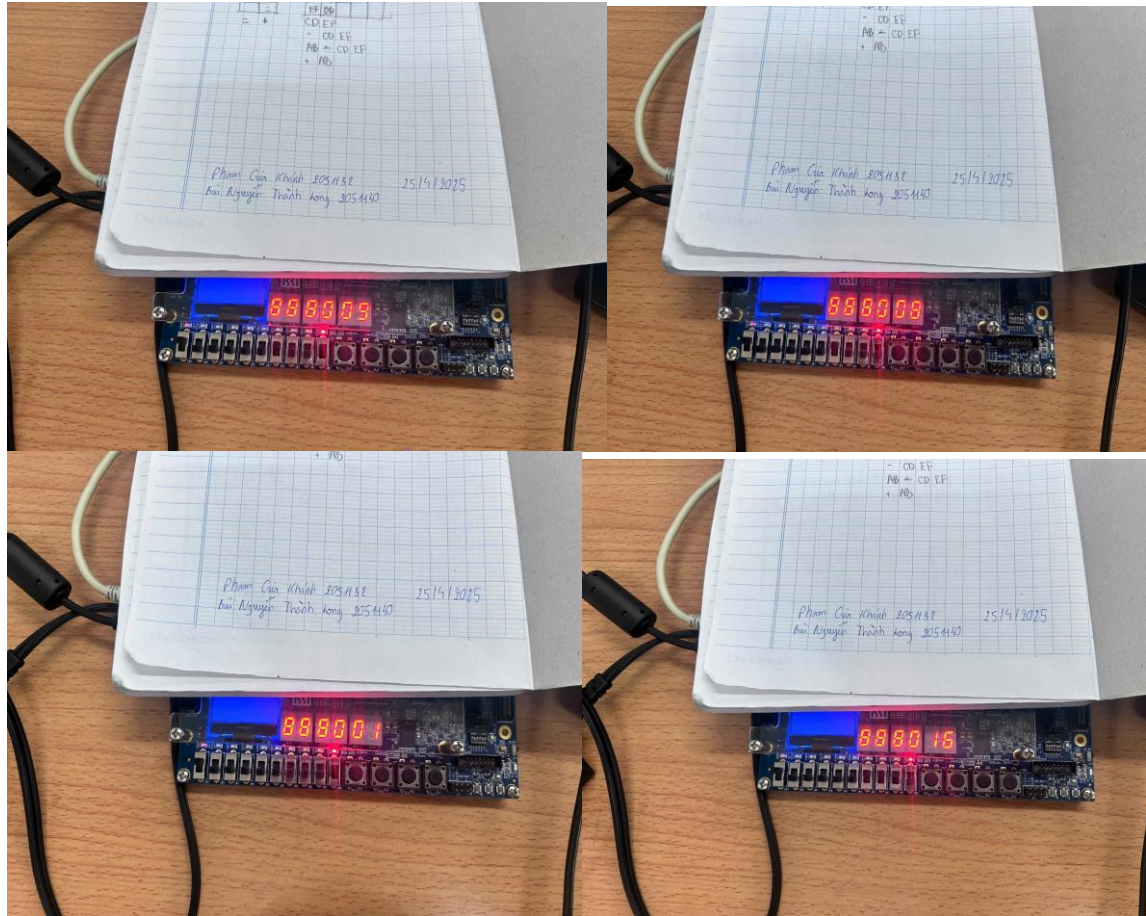
Case 1: $11 + 44 = 55$



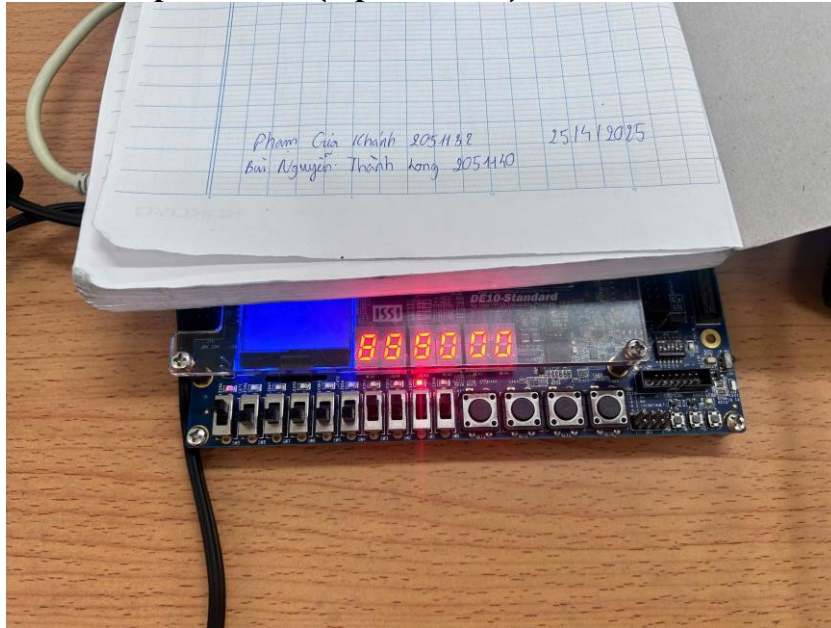
Case 2: $73 - 39 = 34$



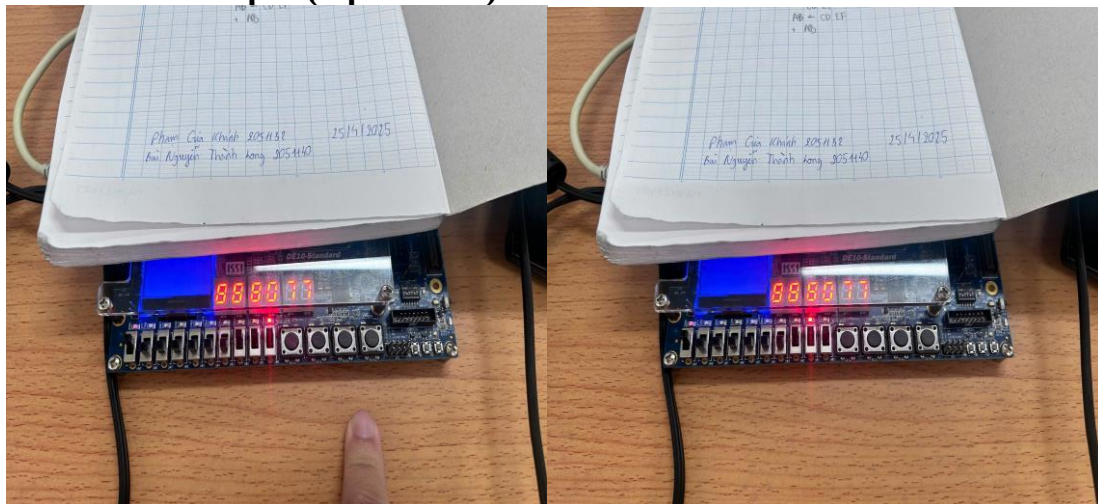
Case 3: $9 + 8 - 1 = 16$



Case 4: Input NULL (input = 1111)



Case 5: Error input (input = 777)



Case 6: Reset Calculator



VI. CONCLUSION

The task involves understanding Reverse Polish Notation (RPN) and its functionality within a stack module provided in stack.sv. The goal is to create a calculator that processes input commands using this stack-based approach, allowing for the implementation of additional functions in Lab5. There are at max 2 operations (only addition, subtraction) and 3 numbers in our calculator. The input number will not be more than 2 digits. The solution involves designing a Finite State Machine (FSM) to handle and process the input and commands sequentially. This approach will enable the creation of a versatile calculator capable of executing various operations efficiently.

VII. REFERENCES

1. Reverse Polish notation

Access from: https://en.wikipedia.org/wiki/Reverse_Polish_notation

2. Stack (abstract data type)

Access from: [https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))